

Multi-agent trading environment

M J Owen, L Lee, G Sewell, S Steward and D Thomas

Agent technology is emerging as an important software engineering paradigm for the 21st century. This paper describes some pioneering work carried out at BT Laboratories on how agents can be applied to the domain of electronic commerce and in particular to the area of advanced trading. The principal benefits and issues are outlined. The paper concludes with a look at future plans to develop additional services.

1. Introduction

The trading environments offered by electronic commerce (eCommerce) will be radically different from the traditional market models in existence today. Users in this new rapidly changing market will require ever more sophisticated services to assist them. Buyers will need to identify and purchase the best from the myriad of products and services available on the Internet; suppliers will want to target potential customers in a more efficient manner, increasing revenue and reducing costs. Software too needs to keep up with rapid change and increasing complexity. The eCommerce market will not tolerate slow response or delayed decision making. New ways of developing channels for interactive relationships with customers and building customer communities must be built [1]. It is this business model which is forcing businesses to take note of software agents.

A software agent is a software entity that can act in an autonomous manner, can learn, is proactive and has the ability to interact with other entities, be they human or software based. Software agents are predicted to play an increasing role in electronic commerce as they exhibit much of the behaviour considered to be desirable or even essential. Agents endowed with these capabilities can significantly reduce the amount of human effort required, therefore providing real value [2].

This paper focuses on how agent technology can be used to support electronic trading in the area of market matching. The remainder of this paper is structured as follows — section 2 highlights the strengths and weaknesses of current approaches to this rapidly developing area, and sections 3—7 describe some research carried out at BT Laboratories (BTL) that has been aimed at developing an intelligent digital shopping assistant intended for the higher value end of the consumer market. It provides buyers with the type of functionality that can assist in purchasing a range of products and services, from cars to customised

travel. The application is described from the perspectives of the purchaser, the merchant and the market provider. The paper concludes with a look at future directions. For a more general discussion on trading environments, the reader is referred to Videlo and Fletcher [3].

2. Market-matching tools

One of the main trends developing in the consumer market is the rapid rise of market-matching tools. These tools facilitate the matching of buyers' requirements for particular products or services with appropriate suppliers. Shopping bots provide significant value to buyers, enabling them to search rapidly across a number of merchants offering the same or similar products, but at different prices. The user can select the merchant offering the product at the cheapest price. These bots are providing a service known as comparison shopping, which works well in particular markets where the products or services are differentiated by one characteristic, say price. Examples of these products include CDs and books [4, 5]. However, while the products can be searched across different features, there is no notion of the system providing any support for the purchaser in the form of evaluating the products against a range of preferences. The buyer is left to make the decision based on a comparison of the price. However, for more expensive products, the purchasing decision generally requires the purchaser to make a complex trade-off between a set of preferences.

Examples of this type of functionality are already emerging on the Internet, with services such as the decision aid system offered by PersonaLogic [6]. However, like many other eCommerce solutions it is client/server based, only providing a front-end to the merchant's product description and therefore biased towards the needs of the merchant rather than the purchaser. Providing an infrastructure in which trusted third parties can advise buyers on purchases will enable users to obtain an

independent perspective by removing any merchant bias. This is an important ingredient of any trusted purchasing assistant that is currently missing from many solutions. It is this capability which will move shopping on the Internet into the next generation. As the product or service increases in value, so the value added to the match-making service significantly increases. For these types of products or services, it is necessary to capture the users' requirements in terms of their preferences for different attributes of the product. The products can then be evaluated against the complete set of preferences. These digital shopping assistants will also offer a number of supplementary services, providing additional information in the form of recommendations from other buyers. This type of service is unique in the virtual world, delivering more information than their physical counterparts. Bookstores are limited by shelf space, and reviews and comments are not available to customers while perusing books. These components, when brought into one's environment, offer a comprehensive shopping experience.

One such approach that goes some way to realising this vision is a prototype, called MATE, that has been developed at BT Laboratories (BTL). The following sections describe the overall architecture and the components of the system — the purchasing adviser, merchant agent and market provider.

3. Multi-agent based trading environment (MATE)

MATE provides a multi-agent based trading environment architecture. The architecture introduces three roles — the end users, or **purchasers**, who use the system to find products from **suppliers** who represent the roles of both retailers and wholesalers, and a **market provider** who provides a brokerage role in matching potential buyers to potential suppliers. Each of these roles is supported by individual agents, whose aims are to achieve the goals of their respective users. This architecture is extensible in that it provides flexibility by allowing new roles such as additional brokers and trusted third parties to be easily added in future versions of this system. The overall architecture is shown in Fig 1.

In this model, suppliers that subscribe to a particular market must agree to abide by a common language, or ontology, which can be used to describe the products within any given market. Since this ontology performs such a central role in the application, it will be described before moving on to describe each of the agents.

4. Ontological issues

4.1 Introduction

For entities in a software system to understand and manipulate the same information, it is necessary to define a format for information exchange. This format essentially has two aspects:

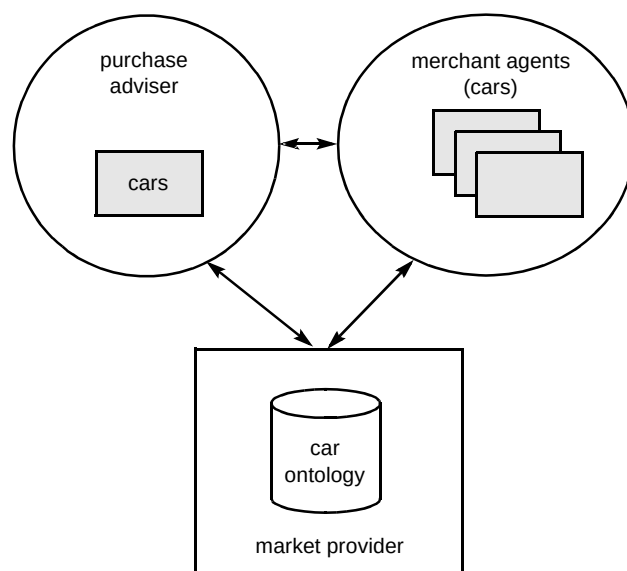


Fig 1 MATE architecture.

- a set of common concepts that are to be understood by all software agents that participate in information exchange — this is the vocabulary, or ontology, of the system and will be familiar to readers who have experience of the design of domain layer knowledge in knowledge based systems (KBSs),
- a syntax to be used to express those concepts.

The market-place is based on the principle of shared product domain ontologies. Product ontologies are used by all the actors taking part in the market-place to describe, match and reason about products. The approach taken in the MATE system was to define a single, simple data format for information exchange that was used to encode both purchaser requirement specifications and merchant product specifications. The use of the same data format for both purposes minimises the complexity of matching a purchaser requirement specification against a set of merchant product specifications.

4.2 Data representation and XML

In creating a data representation format to describe both purchaser requirement specifications and merchant product descriptions, there are two essential characteristics that such a representation should support:

- the representation should be generic — this means that it should allow for easy specification of products and services without including domain-specific constructs within the syntax of the representation,
- the representation should be sufficiently feature-rich to allow for a range of concepts to be modelled.

The extensible mark-up language (XML), defined by the W3C [7], provides an open standard for the

representation of information that can be both processed by machines and read by people. XML has been used in MATE for several purposes. It provides the underlying data structure for:

- purchaser requirement specifications,
- merchant product specifications,
- information passed between the agents in the MATE system.

4.3 A simplified object database schema

The XML specification allows information to be stored in structured files and ported across different platforms. However, unlike conventional *ad hoc* structured file data storage, XML documents can be made to be type-valid against a defined schema or document template definition (DTD). This facility eliminates many of the problems facing distributed systems when addressing interface issues.

For MATE, the approach taken was to define a generic DTD that was used to express a requirement specification or product description in terms of a hierarchical arrangement of objects and attributes. The set of valid objects and attributes for any particular domain, e.g. cars, is contained in an XML file that describes all of the concepts for that domain and can be validated against the DTD. For example, in the case of cars an abstract safety object may contain a brake object, which would have an attribute that describes its type.

A user's preference for a product is represented by the concept of utility that expresses a quantifiable value or worth to that user. As discussed in section 2, the product can be characterised along a single dimension, for example, price. In these cases, the product's utility is measured as a function of the price of the product. However, in many markets, products, such as cars, are described by a number of categories. In these cases, the product utility is defined by a multi-dimensional function, which expresses a complex relationship between the relative importance (weight) given to each category and the worth of the value within the respective category to the user (utility). Within the context of the MATE ontology, weights and utilities are assigned to objects and attributes in the following way.

- Object

Every object may have a 'weight' that expresses the degree of importance a purchaser has associated with that concept. When requirements-matching algorithms are applied to the purchaser's specification, object weights are scaled relative to other object weights at the same level in the object hierarchy.

- Attribute

Every attribute may have a 'utility' that expresses the degree of importance a purchaser has associated with an attribute value. When expressing a preference, the purchaser may have expressed several different possible attribute values that are acceptable for an attribute type. For example, an acceptable colour, modelled by a colour attribute could be any in the set {red, green, blue}. The user may express a preference ordering for these colours. It is also possible to express a degree of dislike that a purchaser wishes to associate with any attribute value.

The model allows both abstract and concrete concepts to be modelled. In the case of cars for example, objects are used to describe abstract concepts such as performance and safety and also more concrete ideas such as brakes, radio, steering column. The manner in which merchant offers are evaluated is dependent on the way in which the multi-dimensional function is represented, and this is discussed in section 5.4.

4.4 Model-view design

The design of MATE adopts a model-view approach by separating domain knowledge information from the graphical representation of that information. In the current implementation, each of the agents has been implemented as a Java application so that they are independent of any domain-specific information. By adopting this approach and producing separate, cross-referenced 'model' and graphic user interface (GUI) product descriptions, it provides a key benefit in flexibility allowing any GUI product description to be downloaded from the market provider at run time. Thus, as new ontologies are added, both the purchase adviser and merchant agent can support new products and services without the need to download a new version of the application.

4.5 Communication

The infrastructure used to support communication between the agents has been built using the ZEUS tool-kit [8]. This will be described in section 7.2.

5. Purchasing adviser

5.1 Introduction

The purchasing adviser supports a buyer during the key stages of the sourcing process (see Fig 2).

Typically, the most difficult parts of the buying process involve understanding, and then comparing, product features. Before a user can enter their preferences for specific product features, they must first understand what

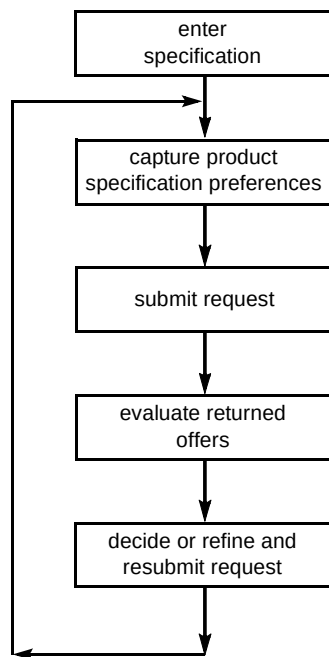


Fig 2 Sourcing process.

product features mean and then decide the relative importance of these features. For example, when buying a car, the buyer must understand what 'ABS' means and then decide if this is a more important requirement than, say, engine power or price. In other words, the buyer must decide if they require a safer, more powerful car or a cheaper car.

Of course, this is not simply a case of picking the single most important attribute, rather all attributes must be assigned relative importance. Understanding a complicated product domain and one's utility function is vital if one is to ensure the best possible purchase. In addition to understanding features and requirements, buyers must also be able to compare different product offerings based on the relative product utilities. The purchasing adviser has been designed to help the buyer at these specification and evaluation stages.

5.2 Architecture

The architecture of the purchasing adviser is shown in Fig 3. The following sections will describe these components in more detail.

5.3 User interface

The purchasing adviser user interface (UI) is created dynamically from an XML specification, which uses information from a particular product ontology. If the buyer wishes to buy a car, the purchasing adviser uses the latest car ontology from the market-place's ontology repository. This method of interface creation has the advantage that it is

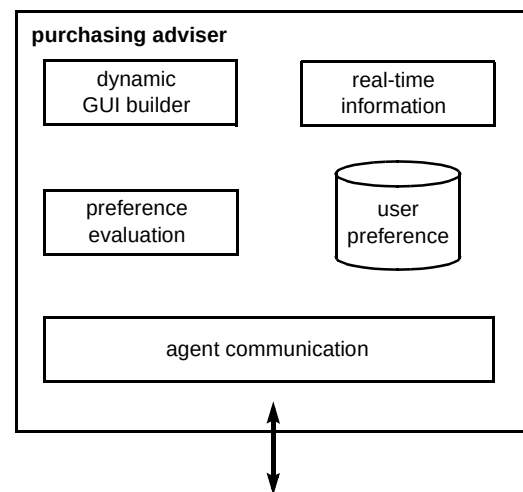


Fig 3 Architecture.

data-driven and therefore the UI will change to reflect corresponding changes in the underlying ontologies. The provision of a range of parsing processes allows different organisations to create differently branded or styled UIs for the same product domain, and to allow different UIs for different input devices including PCs, PDAs or voice.

The user interface specification is written in an interface description language that conforms to an XML graphical user interface DTD. The XML specification of the user interface for any given market must access information from the product ontology. As an example, if the car market is considered, and a GUI component is being specified to capture colour, then the GUI will need to specify all possible values defined within the car product ontology for that object. In the current implementation, this graphical user interface XML specification is created manually, but this will be automated in the next version. The specification is then parsed enabling the user interface to be dynamically created.

The user interface description language defines all of the UI components as described in the previous section.

The user interface is built from a number of basic GUI components including tab menus, text fields, drop down lists, radio buttons and sliders. These basic components are combined to build the following specialised components:

- reservation and maximum price,
- feature selections,
- minimum value.

Figure 4 shows some examples of these GUI components.

In the first example, the user is being requested to enter two values representing the maximum and reservation price.

(a) price

(b) feature selector

(c) minimum

Fig 4 GUI components.

The maximum price, as its name implies, conveys to the merchant the largest amount the buyer is prepared to spend. In contrast, the reservation price is the absolute largest amount the user is prepared to spend, but for which the user wishes to withhold from the merchant. This notion introduces a more general concept of public information, which is given to the merchant, and private information, which is known only to the purchaser but is a key factor used in the evaluation of that attribute or object. This type of information typically includes price but can also include other factors that the purchaser wishes to withhold.

In the second example, the user can select one of the following options:

- don't care,
- won't accept,
- a preference for this feature expressed as a percentage,
- must have.

This user interface provides a rich representation by which the user can precisely express their preference or otherwise for a particular feature.

The third example introduces the idea of a user being able to express the concept of requiring a value to be at least a given minimum value. In the car market, an example of where this occurs is where the buyer may wish to express the requirement that they want the fuel economy to be at least 40 mpg.

It is also possible for the system to suggest likely values and weightings for many of the product attributes. There are various ways to do this. The purchasing adviser could offer the buyer a range of pre-set values based on typical buyer types. The buyer would then review these, making changes where necessary. For example, the buyer would be offered the typical car preferences for 'young family', 'business

motorist' and 'executive'. A more sophisticated method based on user profiling is described in further detail in section 5.5.

The utility specification part of the purchasing adviser ends when the buyer has expressed all their preferences. The buyer's utility values are then encapsulated in a product object and this object is passed to the market-place where it is routed to appropriate merchants. However, in many cases, a buyer may not initially have preferences about all of the attributes of a particular product and therefore their utility may only be partially specified. As an example, the buyer may only express an interest in estate cars within a broad price range. Based on this minimal information, there will be a large number of responses from the merchants. The role of the purchasing adviser is to filter this information down to a meaningful number from which the purchaser can make a sensible decision. There are a number of approaches to resolving this problem. One possible approach that is being pursued in MATE is to guide the user through a sequence of questions prompting the user for additional preferences. The questions will be dynamic in nature reflecting the dependencies on answers to previous questions and the type of merchant responses received. This will lead to more accurate evaluations based on a more precise set of user preferences and hence narrow down the user's choices.

5.4 Evaluating merchant offers

The merchants' agents analyse the buyer's requirements and select the most suitable product offering, based on the merchant's own goals and are then sent back to the purchaser. These offers are collected and evaluated by the buyer's purchasing adviser to produce a ranking based on the preferences captured. It is important to note that the weights and utilities will vary according to each user. Techniques for evaluating preference information about different combinations of product characteristics can be drawn from the domain of multi-attribute utility theory [9, 10] and marketing, such as conjoint analysis [11]. In the current implementation, the ranking is achieved by using an evaluation algorithm that takes three sources of input — the customer's product specification preferences that have been gathered from the user interface, the merchant specifications, and the appropriate evaluation algorithm. In conforming to the overall aim of maintaining genericity in the purchasing adviser, the design of this agent enables the evaluation algorithm to be specified by different organisations and hence override the default algorithms.

The first source comes from the customer's preferences. In the current implementation, each of the three specialised GUI components discussed in the previous section are associated with an evaluation function. As an example, the evaluation function for the price category (see Fig 5) takes the following parameters:

- maximum price given to the merchant, max,
- a maximum utility associated with the maximum value expressed as a value, b , such that $b < 100$ — this allows for merchant specifications to exceed the customer's expectations yet still lie between 0 and 100,
- a reservation price, res, withheld from the merchant,
- an associated utility, c , such that $c < b$.

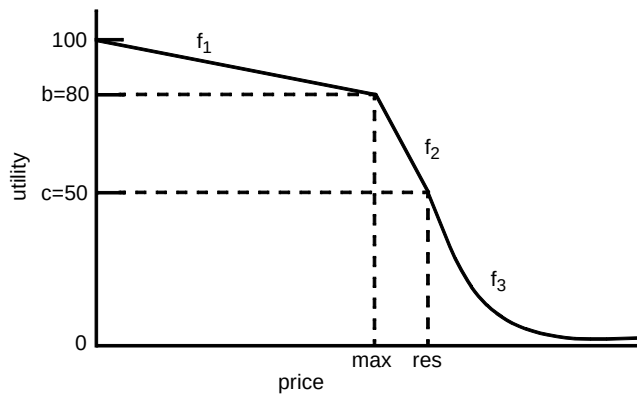


Fig 5 Price.

The evaluation function is defined, such that:

$$u(\text{price}) = \begin{cases} f_1(b, \text{max}) & \text{when } 0 \leq \text{price} \leq \text{max} \\ f_2(b, c, \text{max}, \text{res}) & \text{when } \text{max} \leq \text{price} \leq \text{res} \\ f_3(c, \text{max}) & \text{when } \text{price} \geq \text{res} \end{cases}$$

As can be seen from the above function, the way in which the utility is calculated varies according to whether the price falls within specific ranges. The use of an exponential function for f_3 is used to represent the concept that a buyer's utility will sharply decrease as the price exceeds the reservation price. Utilities corresponding to preferences captured from the other specialised GUI components are evaluated in a similar manner.

Buyers tend to base their purchasing decisions on a set of key categories when purchasing anything other than the simplest types of products. In the case of cars, this set may include price, colour, safety, fuel economy and security. The complexity of consumer buying behaviour comes from the fact that each purchaser assigns relative weights to each of these categories. The purchasing adviser provides a mechanism to capture these relative weightings and uses them to evaluate more precisely the offerings returned from the merchants. The system maps these categories on to the objects and attributes defined in the ontology. As an example, the features category maps on to a number of objects including the sunroof and illumination control objects, whereas the fuel economy category maps directly

on to the fuel attribute. Other categories may include abstract concepts such as performance and safety.

The respective utilities of each category, as defined by the user as pertinent to their purchasing decision, can thus be evaluated. The total utility is derived from using a multi-dimensional function of the weights and utilities. This function can take various forms but a common form used in many domains is that of the linear additive form, which is the approach taken in MATE. In this form, the total utility of a product to a particular purchaser is derived from the following:

$$\text{Total utility}(\text{product}) = \sum_i u_i w_i$$

where u_i and w_i represent respectively the utility and weight of category i .

The user can view each offer in detail to see the features of the products on offer.

5.5 Just-in-time information and user profiling

The personal agent framework (PAF) represents a number of agent-based technologies that have been built at BTL [12]. The MATE project has adapted two of these technologies, RADAR and GRAPEVINE to the domain of electronic commerce. In particular, it offers the buyer the following:

- proactively presents users with information that is relevant to purchase choices that they are making,
- allows users to add and view feedback about products and manufacturers,
- allows users to contact other users who could advise them, based on similar purchases,
- provides default answers for the questions about product specifications.

RADAR's functionality has been adapted to provide information at appropriate points from other sources of information, such as customer feedback forms and consumer reports. As the user moves to different screens, the information content provides a context in which the system can extract key words. By using these key words in conjunction with the user's own user profile, RADAR can refine the search process to display relevant information to assist the buyer. As an example, if the user is currently inputting information relating to safety, and the user's profile indicates preferences suggesting they are interested in purchasing a family car, the system can search the appropriate sources to display information about ABS brakes.

GRAPEVINE enables information to be extracted based on people with similar interests. This has been adapted to take account of buyers who have purchased similar cars, thus providing a mechanism to aggregate this information and hence populate typical preference information that is calculated from previous buyers' choices. For example, the opening question may be car body style. When the buyer chooses 'estate car' the purchasing adviser could suggest: 'Would you like me to suggest the most popular choices for estates cars, from buyers with similar buying patterns to you?' If the buyer agrees then the purchasing adviser will enter values for attributes where possible.

6. Merchant agent

6.1 Introduction

The main objective of the merchant agent is to automate the complex process of matching requirement specifications from purchasers. Merchants will gain significant benefits when the process can select products from their database when there is no exact match but which satisfies both the buyers' broad criteria and the business goals of the merchant. The merchant agent also provides a user interface for creation and modification of product data that is conformant to the product ontology as defined by the market. A key aspect of this user interface is its dynamic nature. Underlying these concepts is the central role played by the model that enables effectively any product or service to be expressed in terms of the requirement-specification representation. The matching process in turn provides the capability to effectively match any product or service.

6.2 Architecture

Figure 6 shows a high-level component view of the architecture of the merchant agent.

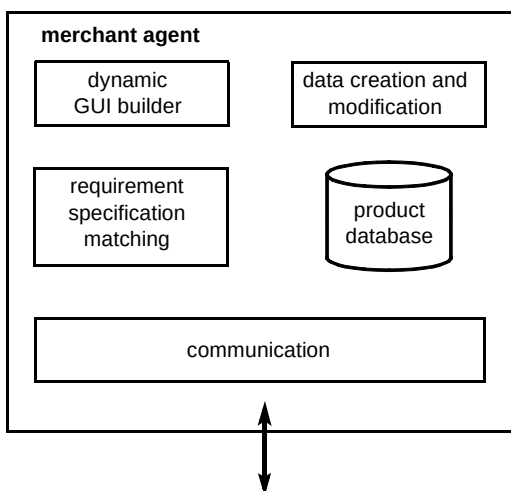


Fig 6 Architecture.

6.3 User interface

The techniques for building the graphical user interface in the merchant agent are similar to the purchasing adviser, and so will not be described again here. Figure 7 shows a montage of typical screenshots from the merchant agent in which the user is creating descriptions of cars that are to be made available to the MATE system.

Additional information, such as image URL and description Web page URL, can be returned to the purchaser adviser.

6.4 Requirement-matching algorithms

As with the development of the requirement-specification representation, the main driving principles behind the design of the requirement-matching algorithms are that they should be generic and extensible. For this reason, the requirement-matching algorithms were written to be capable of matching effectively any product or service that can be expressed in terms of the requirement-specification representation.

Implementation using fuzzy logic

Fuzzy-logic matching [13] is employed so that the merchant agent offers the best match against a purchaser requirement specification, even when no exact match is possible. The algorithm needs to match two types of data, namely textual and numeric data. A product or service is characterised by the text and numbers that represent the values of objects and attributes. Matching is carried out on this data which describes the product or service's specification. As previously described, a purchaser product specification may be characterised by weight and utility values. If a product exhibits a feature that exactly matches a purchaser's requirements for that feature, then the product receives a score equal to the weight or importance the purchaser has attached to that feature. However, if the product's feature is a partial match or is close but not exact, fuzzy rules apply to enable the product to score some fraction of the weight, with that fraction increasing as the match becomes closer.

The algorithms use a word-matching algorithm to match features best described by text, where the score returned is a fraction of the weight specified. For this fraction the numerator is the number of matched words and the denominator the number of words the purchaser specified. For these algorithms, simple string comparison was rejected, as it allows no fuzziness — the strings are either the same or different, with no direct way of testing for some similarity. Since text values are captured by selecting options from a drop down list, there is no opportunity for misspelled or ambiguous words. Therefore character



Fig 7 Product data entry.

matching and tri-gram matching with word preprocessing were discounted, as they generally serve to resolve ambiguities. For numeric features, the units are first checked and converted to the purchaser's units if necessary. Four numeric-matching algorithms are available. The selection of the particular numeric-matching algorithms used depends on whether the purchaser has specified either:

- one preferred value,
- preferred minimum value,
- preferred maximum value,
- range of acceptable values.

In order to allow inexact or fuzzy matching, a 'belief' function has been produced for each of these four situations. The match score is plotted on the y -axis and rises to a maximum of i (the importance) with the product's feature value on the x -axis (purchaser request value = v). Figure 8 illustrates the tent functions for the following algorithms:

- one value specified,
- a preferred minimum,
- a preferred maximum,
- a range (l = preferred low value, h = preferred high value).

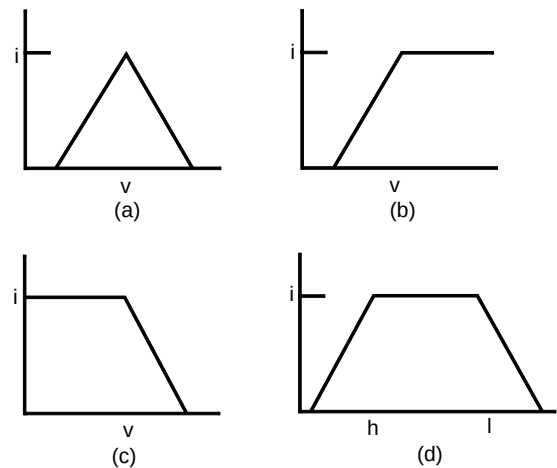


Fig 8 Requirement match fuzzy logic.

The level of fuzziness (or tolerance to partial mismatch) is determined by the gradient of the slope of the function. This gradient is derived in turn from the importance the purchaser has attached to the particular feature. A high importance produces a steep slope and therefore low tolerance to numbers that have a significant deviation from the requested value. As these functions work equally well below the x -axis, a purchaser may specify a negative weight in order to display a degree of dislike for a given attribute. In such a case, any match will receive a negative score (i.e. a score to be subtracted from the product's total) and a partial match in this case will result in a smaller score to be subtracted.

It is also possible to override all the fuzzy matching — for example, to insist that a value lies within a certain range. In such a case for numeric matching all slopes become vertical, and in text matching only all-word matches count.

Another option available is that the purchaser may insist that a certain object or attribute be present in the suggested product. Products not containing that attribute will be 'rejected'. Similarly, a product may be rejected for containing an attribute that the purchaser has insisted is not to be included. However, a merchant may choose to ignore any product rejects and offer the product anyway (if it is otherwise particularly suitable). The converse situation also applies in that any attribute insisted upon by the purchaser and present in the product, or absent and insisted to be absent will be 'accepted', but again the merchant may choose to ignore the product if it mismatches too many other purchaser requests.

In order to arrive at the attributes containing the information, it is necessary to navigate through the attribute's parent objects, for example information on a car's brakes could be contained in a brakes object that could in turn be part of a safety object. Match scores for attributes are scaled according to the weights the purchaser has assigned to their container objects. Should the purchaser decline to enter a weight for an object, then the weight of the parent object will be used, and if the parent object has no weight, a default of half the maximum weight is used. If the purchaser specifies one or more options for an attribute but declines to include weight information, weights will be assigned equidistant from each other and the maximum and minimum possible. A purchaser, who lists four colours, for example, red as his preferred colour, then blue, white and green, but does not specify any weights, will have weights assigned:

- red — 80% of maximum weight,
- blue — 60%,
- white — 40%,
- green — 20%,

The weights are assigned to follow the order in which the colours were entered. This method is one of the least arbitrary ways to auto-assign weights.

In the product data files generated by the merchant agent, a product may be specified with a number of values for a given attribute. For example, several colours of paint may be available. When the best matching attribute value has been determined, then all the other possible values are discarded and the final product specification is presented to the purchaser.

The product may have some features for which the purchaser had specified a preference, which incurs an additional charge. In the final stage of matching, a 'configurator' determines whether to include such features or combinations of features based on whether the decrease

in the match on price is outweighed by the increases on matching elsewhere.

7. Market server

7.1 Introduction

Traditional eCommerce applications are usually developed as either a two-tier or a three-tier distributed system, where Web servers are usually integrated with databases, and other systems to provide these services; while Web browsers are used to collect and view information, and to submit information to complete the transactions. In multi-agent systems, every software agent is regarded as an autonomous entity, which does not simply act as a medium between users and the servers but as a pro-active agent. This agent searches and processes information on the user's behalf, as well as reacts to various opportunities according to the user's preferences.

ZEUS is one such tool-kit that allows developers to build multi-agent based applications [8]. The market server plays the central role of a brokerage agent. The following section will outline how the ZEUS tool-kit has been applied to the development of the MATE system.

7.2 ZEUS tool-kit architecture and approach

The ZEUS tool-kit consists of a set of components that can be categorised into three functional groups as depicted in Fig 9 — a component library, an agent-building tool and a suite of utility agents comprising nameserver, facilitator and visualiser agents.

The component library is a collection of components that form the building blocks of individual agents. They provide the application-independent agent-level functionality such as communication, co-ordination, ontology, and planning. An extensible library of predefined reusable co-ordination protocols, e.g. contract-net and various auction protocols, is also provided.

To facilitate rapid development of multi-agent systems, ZEUS provides a high-level agent visual building tool whereby the development of agents is semi-automated; developers need not know the technical details of ZEUS but rather focus on the applications they want to tackle using the multi-agent approach. In addition to software components, ZEUS also has agent components that provide standard agent-based services. For example, nameservers and facilitators enable information discovery in a distributed system environment, while visualisers help to monitor, control and debug the whole system of agents.

The MATE system is developed using ZEUS. Here, every purchaser and merchant is represented by a ZEUS agent which provides the fundamental services in a

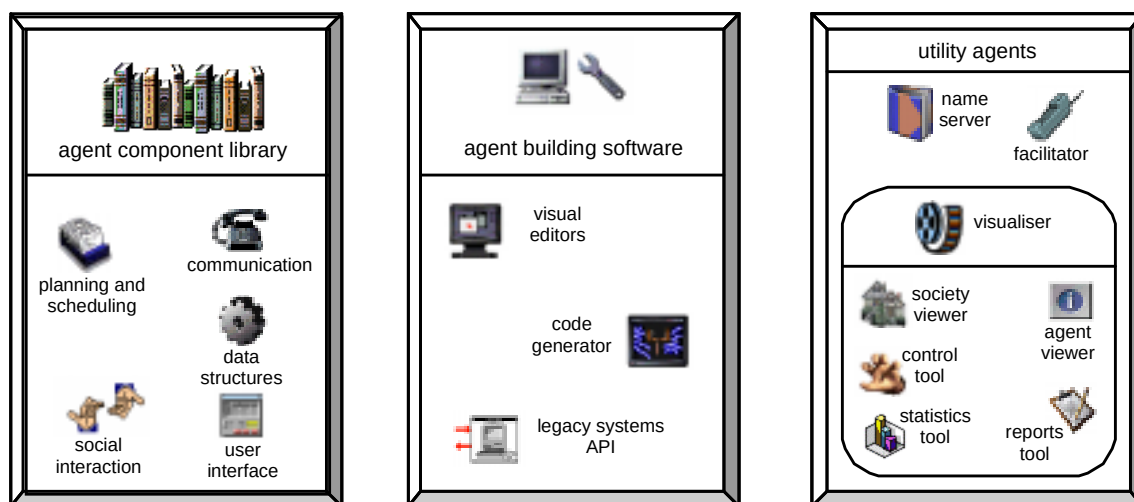


Fig 9 Components of the ZEUS agent-building tool-kit.

distributed system environment — communication, agent co-ordination, and task co-ordination. ZEUS regulates the search of (product) information and ensures the consistency, integrity and the completeness of transaction handling. Nameservers are used to serve as a ‘yellow pages’ facility so that the purchasing agent is able to find the appropriate merchant agents.

8. Conclusions

This paper has shown how agent technology offers a new approach to handling the complex nature of eCommerce. The MATE architecture takes a radical new approach and allows us to view eCommerce from a different perspective. The framework provides an extensible platform upon which a number of other components can be built. Future research is now needed to extend the components on this platform to provide a richer set of brokerage functionality. These components will enable MATE to support a more diverse market space and provide significant added value to the parties involved. In providing a market-matching capability, the platform only supports one aspect of a service. The ability to offer a complete end-to-end process would enable a much wider range of services to be offered. A key component of extending the capabilities would be to define a service ontology that describes how a high-level service is decomposed into a number of sub-service components. By providing this functionality and the ability to plan, schedule and automate the negotiation process, the system has the potential to offer more complex services in which the user can express their requirements as a higher level goal.

An example of where this may be applied is in the travel industry, where users only need to request a business trip subject to a maximum cost and other preferences. The system is then able to break down the service into a number

of smaller components, each of which can be negotiated subject to the constraints imposed.

In conclusion, agent technology provides a major new engineering paradigm that businesses must understand and use if they are to successfully meet the challenges of building the next generation of eCommerce solutions.

Acknowledgements

The authors would like to thank colleagues within BT for their support in the production of this paper, in particular Meena Haram from BT Mahindra and David Andrews from the Electronic Commerce Research group.

References

- 1 Steward S, Callaghan J and Rea T: ‘The eCommerce revolution’, *BT Technol J*, **17**, No 3, pp 124—132 (July 1999).
- 2 Owen M J and Nunez-Suarez J: ‘Agent based solutions for eCommerce’, *British Telecommunications Eng J*, **17**, No 4, pp 237—244 (1999)
- 3 Videlo I D E and Fletcher A M: ‘Trading environments’, *BT Technol J*, **17**, No 3, pp 24—32 (July 1999).
- 4 Acses. <http://www.acses.com>
- 5 Amazon. <http://www.amazon.com>
- 6 PersonaLogic. <http://www.personalogic.com>
- 7 XML. <http://www.w3.org/XML/>
- 8 Collis J C, Ndumu D T, Nwana H S and Lee L C: ‘The ZEUS agent building tool-kit’, *BT Technol J*, **16**, No 3, pp 60—68 (July 1998).
- 9 Luce R D and Raiffa H: ‘Games and Decisions’, John Wiley & Sons, Harvard University (1967).

- 10 Raiffa H: 'The art and science of negotiation', Harvard University Press (1982).
- 11 Green P E and Srinivasan V: 'Conjoint analysis in consumer research — issues and outlook', J of Consumer Research (1978).
- 12 Soltysiak S J and Crabtree I B: 'Automatic learning of user profiles — towards the personalisation of agent services', BT Technol J, 16, No 3, pp 110—117 (July 1998).
- 13 Zadeh L A (Ed): 'Fuzzy Logic for the Management of Uncertainty', John Wiley & Sons (1992).



Martin Owen currently leads a team investigating the application of agent-based solutions in eCommerce at BT Laboratories. He gained an MSc in mathematics and modern control systems in 1974. After a period in teaching, he was awarded an MSc in Computer Science in 1991 from the University of Essex, focusing on artificial intelligence and formal methods.

He joined BT Laboratories in 1991. In 1994, he moved to the field of agent technology. He has led a number of European projects and currently leads one that is driving and validating the FIPA agent standards. He commenced a part-time PhD in the use of automated negotiation and agent technology at QMW, University of London in 1995. He is a member of the BCS and a Chartered Engineer.



Lyndon Lee received an MSc in computer science (artificial intelligence) with distinction and a PhD in computer science from the University of Essex in 1989 and 1996 respectively. His doctoral thesis was in a formal model of multi-agent negotiation. From 1992-5 he was a recipient of the Croucher Foundation scholarship, and between 1987 and 1992 he worked as a developer and a consultant with a number of companies. After a short-term research fellowship with BT Laboratories, he joined the Intelligent Systems Research group in 1996, carrying out research and development

of real-world multi-agent systems. In 1997, the ZEUS project that he co-developed at BTL won the BCS IT award and since then has four patents on application. His research interests lie primarily in distributed artificial intelligence, artificial life, decision theory, game theory, distributed and nonlinear systems. He is a member of the BCS.



and trading environments.

Simon Steward joined BT in 1984 as an apprentice at BT Laboratories. After his apprenticeship he joined the submarine optics division where he worked on supervisory and control techniques for laser amplifier repeater systems. He graduated from the University of Warwick in 1991 with a BEng in electronic engineering.

Since returning to BT Laboratories he has worked in the cybernetics group where he studied self-organising systems, and recently in the electronic commerce research group investigating emerging new media markets



Greg Sewell holds a BSc in Physics. He joined BT in 1993 working in the field of network design. Subsequently, he joined a software team working in building tools to support the pricing of networks for corporate clients.

For the last two years he has been involved in building small-scale systems based on newer software technologies (especially Java and CORBA) with a view to downstreaming these technologies and skills into the business.

He is currently completing an MSc in computer science with the Open University.



Dave Thomas joined BT in 1997 after completing his masters degree in astrophysics at Cardiff University.

He has spent the last 15 months working for Cardiff Software Engineering Centre's network computing applications team. During that time he has designed and written several intelligent matching engines for use in eCommerce, including the merchant agent matching engine in the MATE project.

He is currently studying for a degree in Computing for Commerce and Industry at the Open University.